Chapter 6

Association Analysis (Part 2)

Overview

- The FP-growth algorithm
- Interestingness (association rule evaluation)
- Mining quantitative association rules
- Mining sequential patterns

- Reading assignment:
  - Pages 451-473

FP-Growth

- An efficient algorithm for mining frequent itemsets, alternative to Apriori algorithm
- (a) Compress a large database of transactional records into a compact *Frequent-Pattern Tree* (*FP-tree*) structure
  - Store the structure in memory to avoid costly database scans ← I/O efficient
- (b) Run an efficient algorithm to extract frequent itemsets from the FP-tree.
  - A divide-and-conquer recursive strategy
  - Avoid candidate generation and subset testing ← CPU efficient

FP-tree Construction

Let $\rho_s = 0.6 \Rightarrow$
min_support = 3

Item	*frequency*
f	*4*
c	*4*
a	*3*
b	*3*
m	*3*
p	*3*

TID	Items bought	(ordered) frequent items
100	{f, a, c, d, g, i, m, p}	{f, c, a, m, p}
200	{a, b, c, f, l, m, o}	{f, c, a, b, m}
300	{b, f, h, j, o}	{f, b}
400	{b, c, k, s, p}	{c, b, p}
500	{a, f, c, e, l, p, m, n}	{f, c, a, m, p}

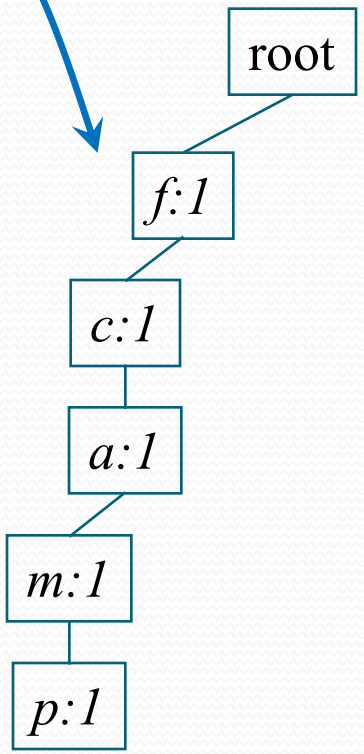
Steps:

1. Scan DB once, find frequent 1-itemsets (single item patterns)
2. Remove infrequent items from transactions
3. Order frequent items in each transaction in descending order of their frequency
4. Scan (the reduced) DB again, construct FP-tree

FP-tree Construction

TID	freq. Items bought
100	{f, c, a, m, p}
200	{f, c, a, b, m}
300	{f, b}
400	{c, b, p}
500	{f, c, a, m, p}

derive a tree path for each transaction



min_support = 3

Item	*frequency*
f	*4*
c	*4*
a	*3*
b	*3*
m	*3*
p	*3*

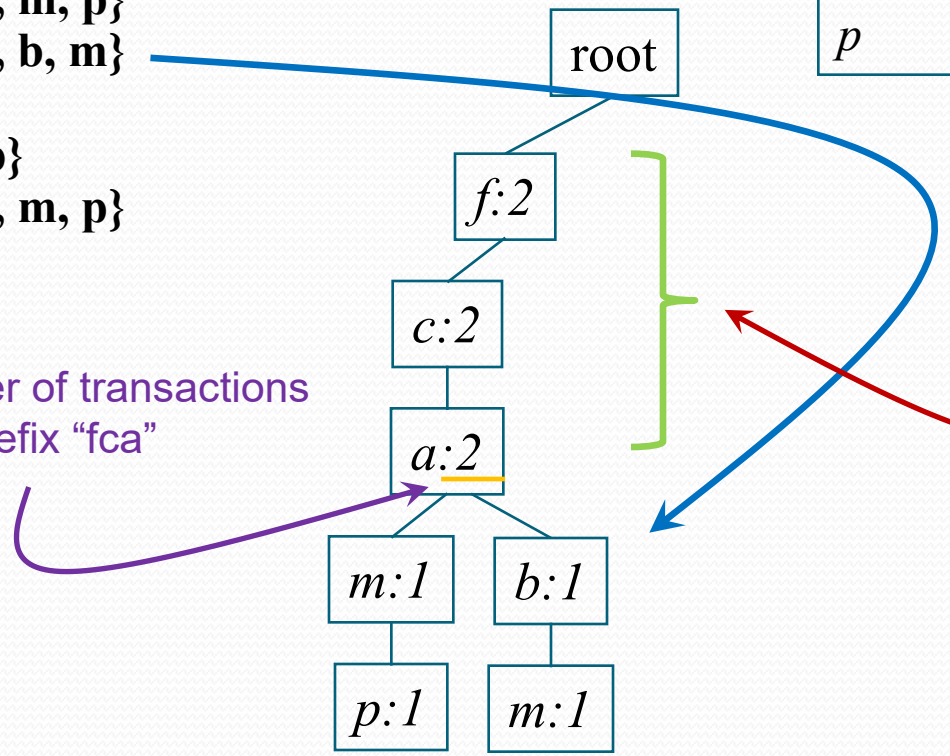
Each node represents a "prefix",
which is given by the path from the root to the node.

FP-tree Construction

min_support = 3

Item	*frequency*
f	*4*
c	*4*
a	*3*
b	*3*
m	*3*
p	*3*

TID	freq. Items bought
100	{f, c, a, m, p}
200	{f, c, a, b, m}
300	{f, b}
400	{c, b, p}
500	{f, c, a, m, p}



Number of transactions with prefix “fca”

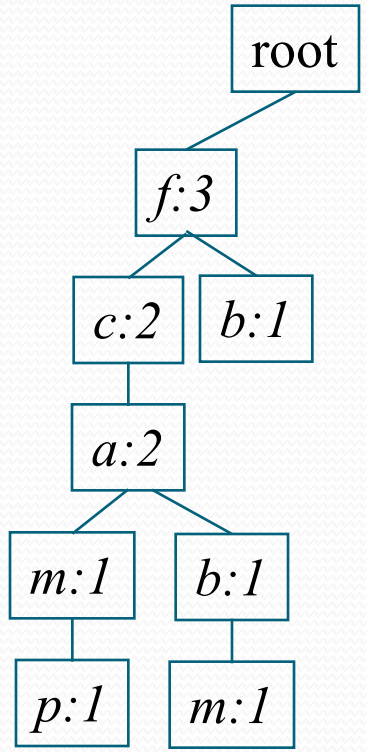
Common prefixes are merged

FP-tree Construction

TID	freq. Items bought
100	{f, c, a, m, p}
200	{f, c, a, b, m}
300	{f, b}
400	{c, b, p}
500	{f, c, a, m, p}

min_support = 3

Item	*frequency*
f	*4*
c	*4*
a	*3*
b	*3*
m	*3*
p	*3*



FP-tree Construction

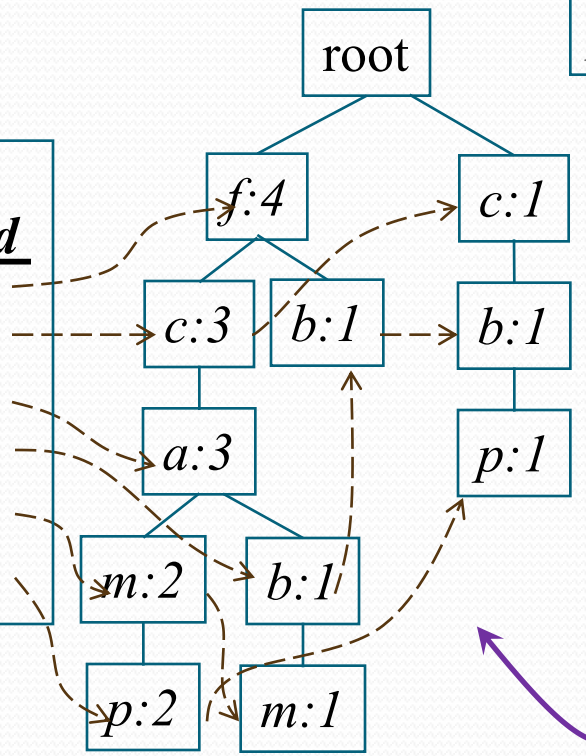
TID	freq. Items bought
100	{f, c, a, m, p}
200	{f, c, a, b, m}
300	{f, b}
400	{c, b, p}
500	{f, c, a, m, p}

min_support = 3

Item	*frequency*
f	*4*
c	*4*
a	*3*
b	*3*
m	*3*
p	*3*

Header Table

Item	*frequency*	*head*
f	*4*	
c	*4*	
a	*3*	
b	*3*	
m	*3*	
p	*3*	



Nodes with the same item label form a linked list

This structure is more compact than the original transactional DB if customers buy similar items (i.e., with long common prefixes).

Mining Frequent Patterns Using FP-tree

- General idea (divide-and-conquer)
  - Recursively grow frequent pattern (itemsets) path using the FP-tree
- Method
  - For each item, construct its *conditional pattern-base*, and then its *conditional FP-tree*
  - Repeat the process on each newly created conditional FP-tree
  - Until the resulting FP-tree is empty, or it contains only one path. (From the single path, each possible item subset derived from it is a frequent pattern.)

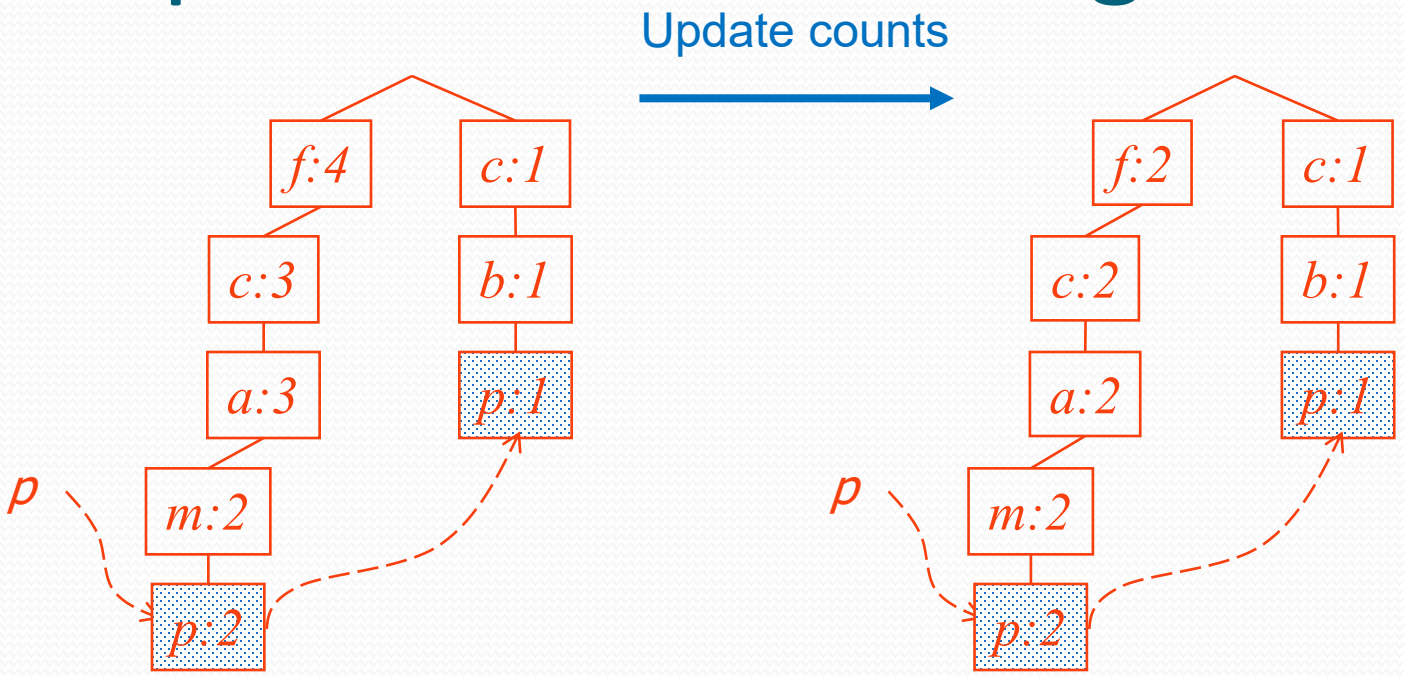
------- p	------- m	------- b	------- a	------- c	------- f
------- p	------- m	------- b	------- a	------- c	------- f
------- p	------- m	------- b	------- a	------- c	------- f
------- p	------- m	------- b	------- a	------- c	------- f
….	….	….	….	….	….

Divide the problem into 6 sub-problems, each one discovers frequent patterns that end with a specific item (p, m, b, a, c, f).

Mining Frequent Patterns Using FP-tree

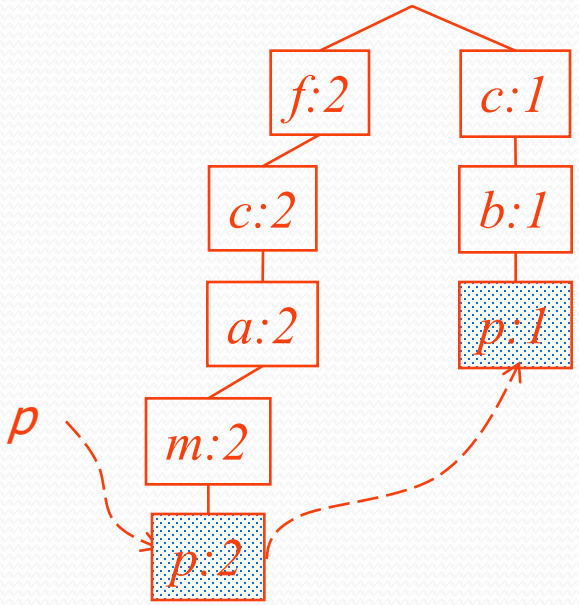
- Start with last item in order (i.e., *p*).
- Objective: find all patterns that end with item "*p*".
- Follow node pointers starting from the header table and extract the part of the tree that is relevant to "*p*".
- Accumulate all transformed prefix paths of that item to form a conditional pattern base.
- Recursively solve the mining problem for the conditional pattern base and concatenate each frequent itemsets found in the conditional pattern base with *p*.

Mining Frequent Patterns Using the FP-tree



We have 2 [fcam | p] patterns
and 1 [cb | p] pattern

Mining Frequent Patterns Using the FP-tree



TID	Items bought	(ordered) frequent items
100	{f, a, c, d, g, i, m, p}	{f, c, a, m, p}
200	{a, b, c, f, l, m, o}	{f, c, a, b, m}
300	{b, f, h, j, o}	{f, b}
400	{b, c, k, s, p}	{c, b, p}
500	{a, f, c, e, l, p, m, n}	{f, c, a, m, p}

If this were the dataset, what would be the FP-tree?

Conditional* pattern base for *p

fcam:2, cb:1

{f, c, a, m}
{f, c, a, m}
{c, b}

p

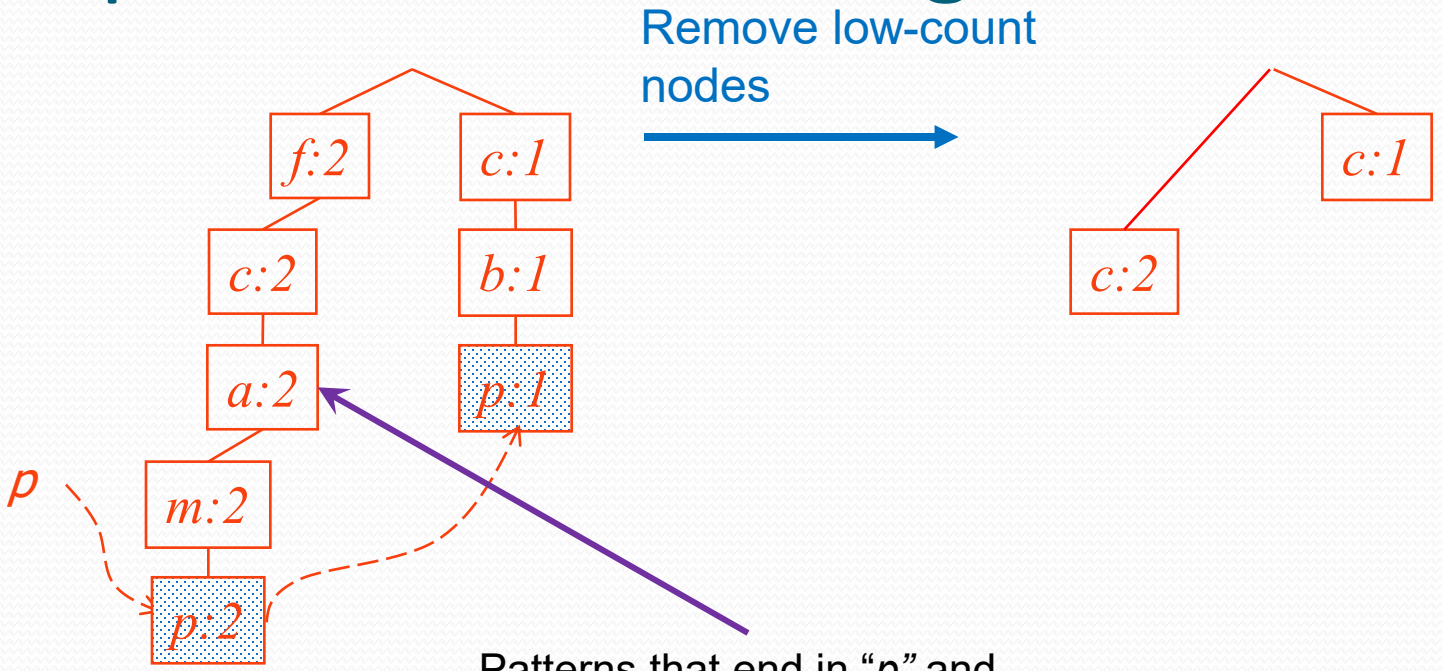
{c}
{c}
{c}

p

Construct a new FP-tree based on this conditional pattern base by keeping nodes that appear $\geq \rho_s \times N$ times and merging paths.

This leads to only one branch *c:3*
Thus, we derive only one frequent pattern containing *p*: pattern *cp* with support 3

Mining Frequent Patterns Using the FP-tree



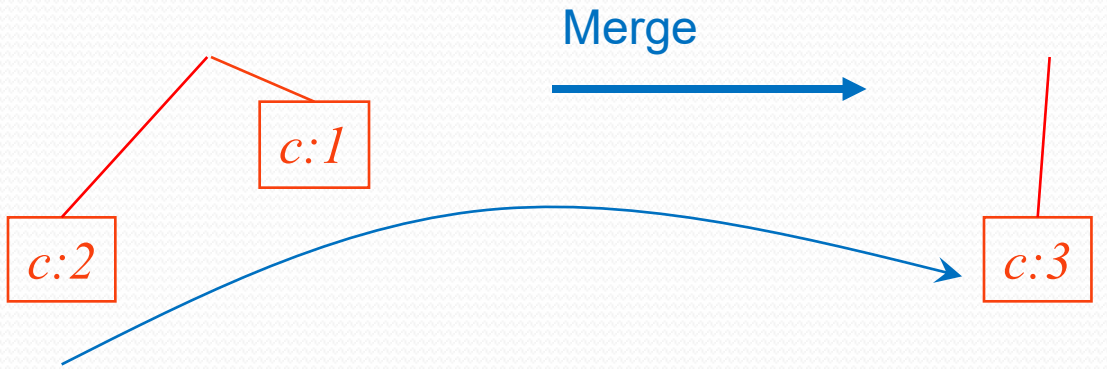
Patterns that end in "*p*" and involve "*a*" are too few (< 3), so, "*a*" is discarded.

Likewise, for "f", "m", "b"

{c}
{c}
{c}

p

Mining Frequent Patterns Using the FP-tree



Just one path left => base case.
Enumerate all possible subsets.
This gives $\emptyset, \{c\}$.

- Resulting FP-tree conditional on *p*. If we extracted all the transactions that involve *p* and built that frequent pattern tree, this is what we would get. No need to scan the DB to obtain it.
- Frequent patterns: *p*, *cp*

Mining Frequent Patterns Using the FP-tree (cont'd)

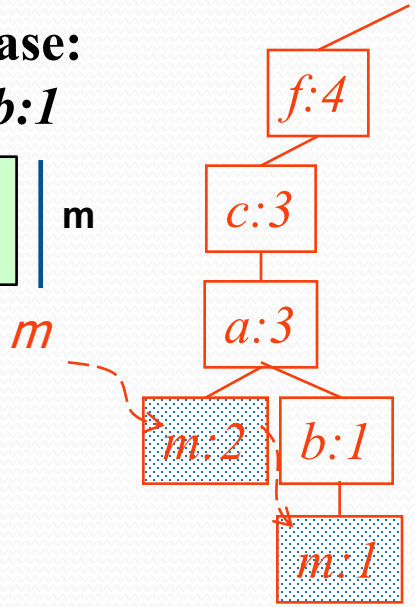
- Move to next least frequent item in order, i.e., *m*
- Follow node pointers and traverse only the paths containing *m*.
- Accumulate all transformed prefix paths of that item to form a conditional pattern base
- Recursively solve mining problem for conditional pattern base and concatenate frequent itemsets with *m*

Mining Frequent Patterns Using the FP-tree (cont'd)

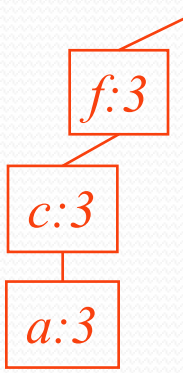
***m-conditional* pattern base:**
fca:2, fcab:1

{f, c, a}
{f, c, a}
{f, c, a, b}

m



update counts



***m-conditional* FP-tree (contains only path *fca:3*)**

All frequent patterns that include *m*

m,

fm, cm, am,

fcm, fam, cam,

fcam

Conditional Pattern-Bases for the example

Item	Conditional pattern-base	Conditional FP-tree
p	{(fcam:2), (cb:1)}	{(c:3)}|p
m	{(fca:2), (fcab:1)}	{(f:3, c:3, a:3)}|m
b	{(fca:1), (f:1), (c:1)}	Empty
a	{(fc:3)}	{(f:3, c:3)}|a
c	{(f:3)}	{(f:3)}|c
f	Empty	Empty

FP Growth summary

- Advantages
  - No candidate generation, no subset testing
  - Uses compact data structure
  - Eliminates repeated database scan
  - Basic operation is counting and FP-tree building
  - Usually more efficient than Apriori, especially when there are many long patterns
- Disadvantages
  - Large memory requirement to store the whole tree.

Pattern Evaluation

- Association rule algorithms tend to produce too many rules
  - Many of them are uninteresting or redundant
  - $\{A,B,C\} \rightarrow \{D\}$ vs. $\{A,B\} \rightarrow \{D\}$ are redundant if they have similar support & confidence
  - $\{A,B,\} \rightarrow \{D\}$ vs. $\{A,B\} \rightarrow \{D,E\}$

Which rule is better?

- Interestingness measures can be used to *prune* or *rank* the derived patterns
- In the original formulation of association rules, *support* & *confidence* are the only measures used

Drawback of Confidence

	Coffee	$\overline{\text{Coffee}}$	
Tea	15	5	20
$\overline{\text{Tea}}$	75	5	80
	90	10	100

Association Rule: Tea $\rightarrow$ Coffee

Confidence= P(Coffee|Tea) = 0.75

How good is this rule?

Drawback of Confidence

	Coffee	$\overline{\text{Coffee}}$	
Tea	15	5	20
$\overline{\text{Tea}}$	75	5	80
	90	10	100

Association Rule: Tea $\rightarrow$ Coffee

Confidence= P(Coffee|Tea) = 0.75

but P(Coffee) = 0.9

$\Rightarrow$ Although confidence is high, rule is misleading

$\Rightarrow$ P(Coffee|$\overline{\text{Tea}}$) = 0.9375

Lift

- Given a rule $X \rightarrow Y$
- A measure that considers statistical dependency

$$Lift = \frac{P(Y|X)}{P(Y)} = \frac{P(X \wedge Y)}{P(X)P(Y)}$$

joint probability of x and y

probability of "x and y" if they are independent

Lift > 1 => *x* and *y* are positively associated.
Lift < 1 => *x* and *y* are negatively associated.

Example: Lift

	Coffee	$\overline{\text{Coffee}}$	
Tea	15	5	20
$\overline{\text{Tea}}$	75	5	80
	90	10	100

Association Rule: Tea $\rightarrow$ Coffee

Confidence= P(Coffee|Tea) = 0.75

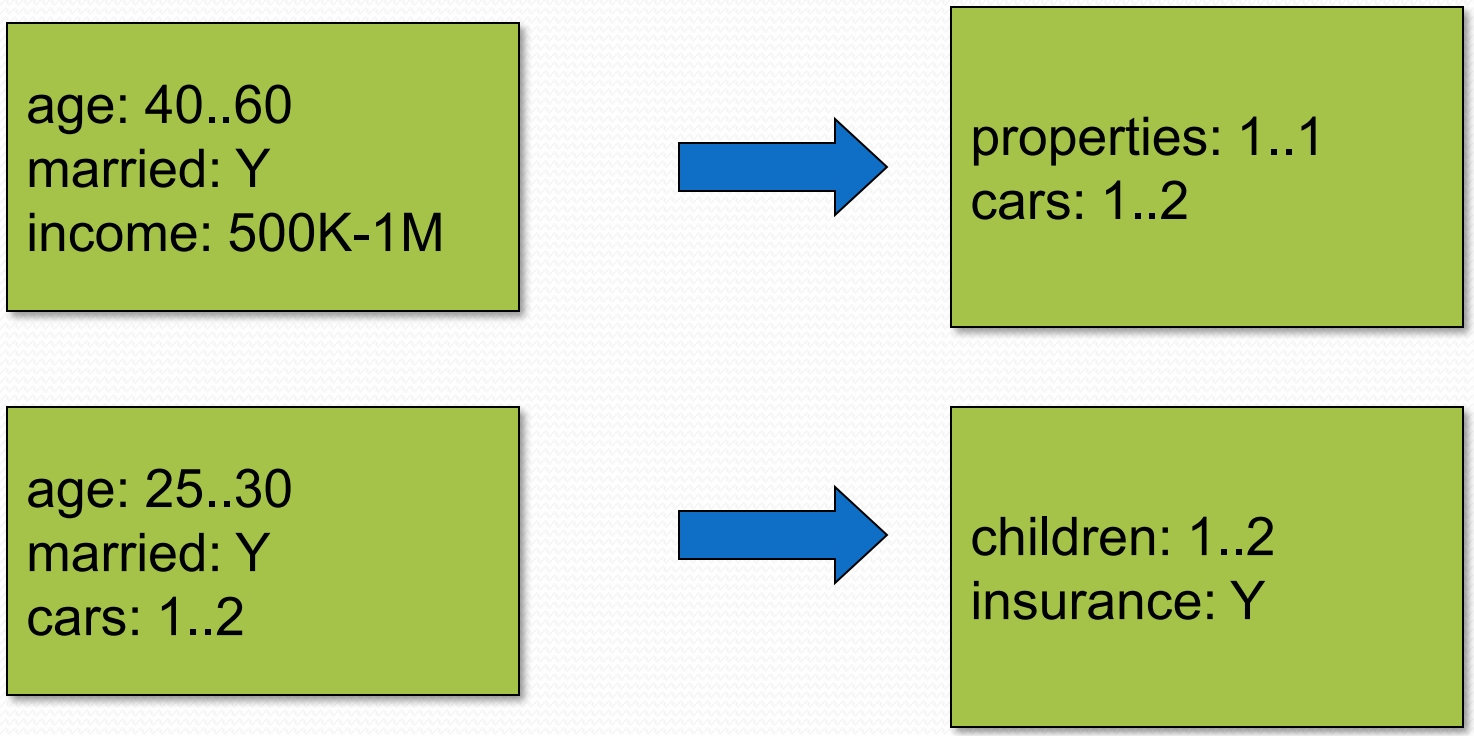
but P(Coffee) = 0.9

$\Rightarrow$ Lift = 0.75/0.9= 0.8333 (< 1, therefore Coffee and Tea are negatively associated)

Quantitative Association Rules (QAR)

- In binary association rule mining, attributes are assumed to be binary (0/1).
- In practice, many datasets contain *quantitative* attributes.
- Example: a dataset of customer records may record information like “age”, “income”.
- These attributes are quantitative (i.e., they take on numerical values).
- Quantitative association rules deal with, not only binary attributes, but also quantitative attributes.

Examples


age: 40..60
married: Y
income: 500K-1M
properties: 1..1
cars: 1..2
age: 25..30
married: Y
cars: 1..2
children: 1..2
insurance: Y

example dataset D

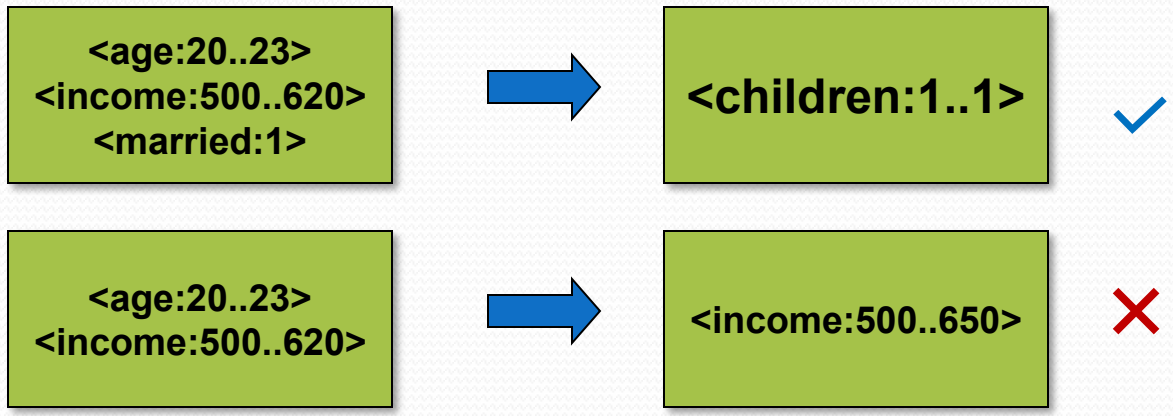
Record ID	Age	Income (K)	married	children
1	20	250	N	0
2	22	300	N	0
3	24	300	Y	0
4	26	290	Y	0
5	28	400	N	0
6	30	350	Y	1
7	32	700	N	0
8	34	750	Y	1
9	36	1,000	Y	2
10	38	1,000	N	0
11	20	300	N	0
12	22	280	Y	1
13	24	290	N	0
14	26	340	N	0
15	28	800	Y	2
16	30	350	Y	1
17	32	370	Y	1
18	34	350	Y	0
19	36	450	Y	1
20	38	750	Y	2
21	39	900	Y	2
22	36	600	N	0
23	35	900	N	0
24	37	900	Y	2
25	34	650	Y	2

Problem Definition

- based on Srikant and Agrawal
- Given a dataset D, define the set I_R as one that contains 2 kinds of elements:
  - $<x: l..u>$, where x is a quantitative attribute; l and u define a range of x.
  - $<x: v>$, where x is a binary attribute; v is either 1 or 0 (or "yes/no")
- E.g., <age: 20..23>, <income: 500..620>, <married: 1>, <children: 1..1> are example elements of I_R.

Problem Definition

- A QAR is an implication rule of the form:
  $X \Rightarrow Y$,
  where X and Y are subsets of I_R and that no attribute appears in both X and Y.
- Example: which one of the following is a QAR?



Support and Confidence

- Given a set $X \subseteq I_R$, a record *r* in the dataset supports *X* if *r* contains values that fall into the corresponding ranges of the attributes mentioned in *X*.
- E.g., if *X* = {<age: 30..35>, <income: 500..800>, <married: 1>}, then only records 8 and 25 in *D* support *X*.
- E.g., Record 18 does not support *X* because the value of "income" is 350, which is outside the range [500..800].

Support and Confidence

- Similar to binary association rules, our goal is to find all rules
  $X \Rightarrow Y$
  such that the support and confidence conditions are satisfied.
- That is:
  - $\sup(X \cup Y) / N \geq \rho_s$
  - $\sup(X \cup Y) / \sup(X) \geq \rho_c$

QAR

- Mining QAR is a lot more expensive than mining binary association rules because the *number of items* considered under QAR is much larger.
- Each (discrete) numerical attribute with a domain of t values derives $O(t^2)$ items in I_R.
  - Example: Let *Age*'s domain be [0 … 99] (t = 100)
  - How many possible ranges (or <*age*: *l*..*u*>) can be derived?
    (A: for each choice of *l*, there are 100-*l* choices of *u*.
    So, the total number of ranges = $\sum_{i=0}^{99}(100 - i) = 5{,}050$

Mapping QAR to the binary model

- Our approach of finding QARs is to
  - map the dataset with quantitative attributes to one with binary attributes, then
  - apply Apriori on the binary dataset.
  - We discretize each quantitative attribute into intervals, each then derives a binary attribute using 1-hot encoding.

Transforming Attributes

- Each quantitative attribute is transformed into *n* binary attributes, where *n* is the number of intervals created by discretization.
- Example:

…	age	…
…	28	…

…	<age:20..24>	<age:25..29>	<age:30..34>	<age:35..39>	…
…	0	1	0	0	…

record id	Age:20..24	Age:25..29	Age:30..34	Age:35..39	Inc:250..499	Inc:500-749	Inc:750..1000	married	children:0..0	children:1..1	children:2..2
1	1	0	0	0	1	0	0	0	1	0	0
2	1	0	0	0	1	0	0	0	1	0	0
3	1	0	0	0	1	0	0	1	1	0	0
4	0	1	0	0	1	0	0	1	1	0	0
5	0	1	0	0	1	0	0	0	1	0	0
6	0	0	1	0	1	0	0	1	0	1	0
7	0	0	1	0	0	1	0	0	1	0	0
8	0	0	1	0	0	0	1	1	0	1	0
9	0	0	0	1	0	0	1	1	0	0	1
10	0	0	0	1	0	0	1	0	1	0	0
11	1	0	0	0	1	0	0	0	1	0	0
12	1	0	0	0	1	0	0	1	0	1	0
13	1	0	0	0	1	0	0	0	1	0	0
14	0	1	0	0	1	0	0	0	1	0	0
15	0	1	0	0	0	0	1	1	0	0	1
16	0	0	1	0	1	0	0	1	0	1	0
17	0	0	1	0	1	0	0	1	0	1	0
18	0	0	1	0	1	0	0	1	1	0	0
19	0	0	0	1	1	0	0	1	0	1	0
20	0	0	0	1	0	0	1	1	0	0	1
21	0	0	0	1	0	0	1	1	0	0	1
22	0	0	0	1	0	1	0	0	1	0	0
23	0	0	0	1	0	0	1	0	1	0	0
24	0	0	0	1	0	0	1	1	0	0	1
25	0	0	1	0	0	1	0	1	0	0	1

Finding frequent itemsets

- After the mapping, each attribute-interval is considered a binary item. Apriori is then applied on the binary dataset to find all the frequent itemsets.

Frequent itemsets

- Example: if we set ρ_s = 20%, then the frequent itemsets found in D are:

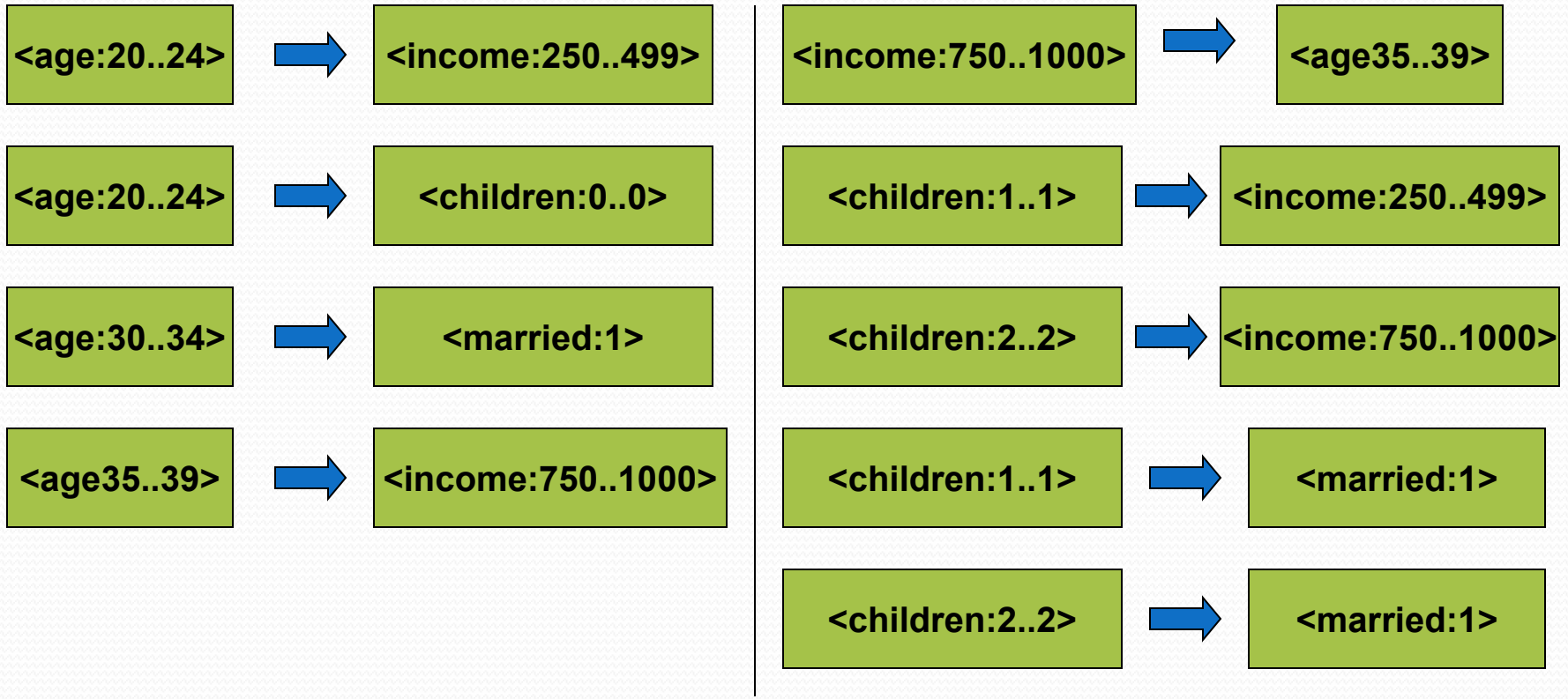
Itemset	Support
<age:20..24>	6
<age:30..34>	7
<age:35..40>	8
<income:250..499>	14
<income:750..1000>	8
<married:1>	15
<children:0..0>	13
<children:1..1>	6
<children:2..2>	6

Frequent itemsets

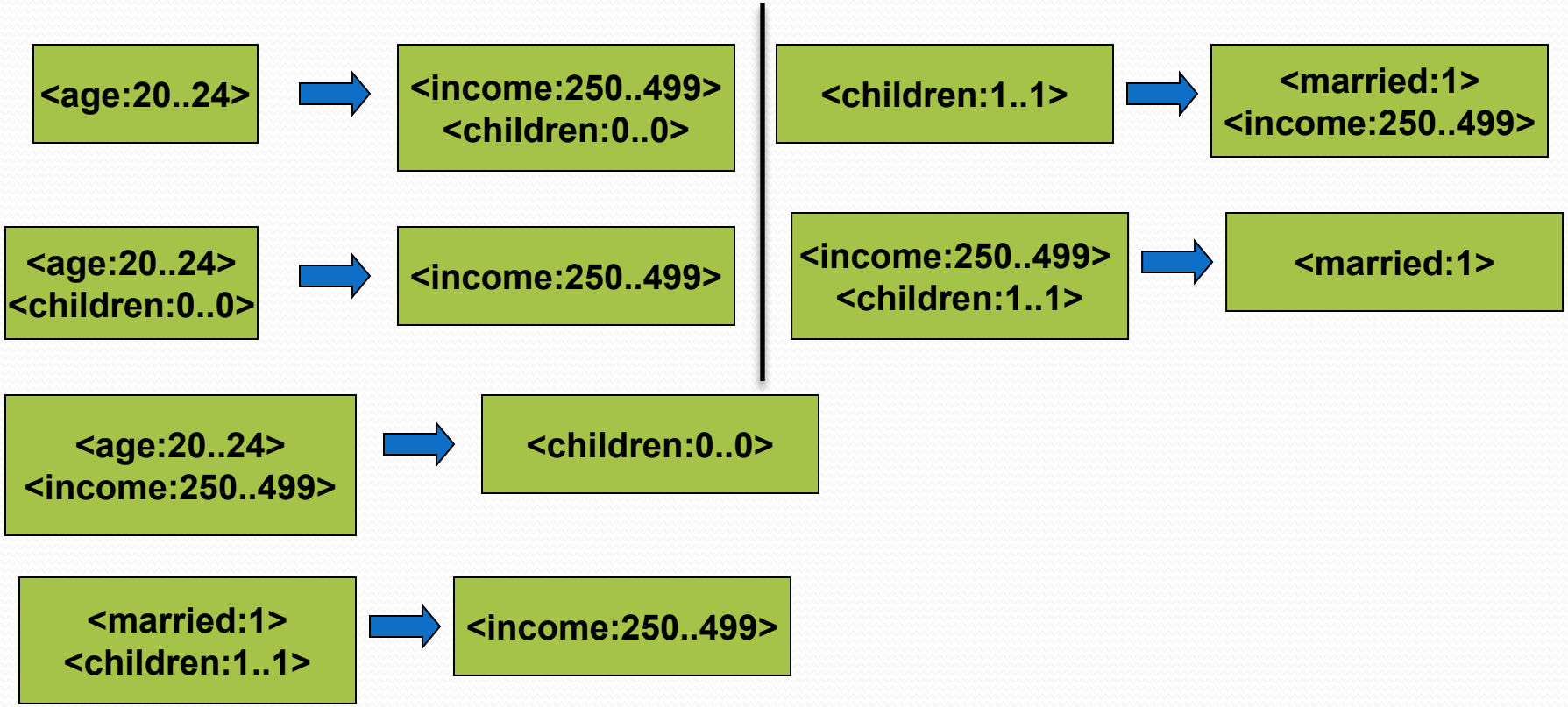
Itemset	Support
<age:20..24>, <income:250..499>	6
<age:20..24>, <children:0..0>	5
<age:30..34>, <married:1>	6
<age:35..39>, <income:750..1000>	6
<age:35..39>, <married:1>	5
<income:250..499>, <married:1>	8
<income:250..499>, <children:0..0>	9
<income:250..499>, <children:1..1>	5
<income:750..1000>, <children:2..2>	5
<married:1>, <children:1..1>	6
<married:1>, <children:2..2>	6
<age:20..24>, <income:250..499>, <children:0..0>	5
<income:250..499>, <married:1>, <children:1..1>	5

Rules

- Example: if we set ρ_c = 70%, then 15 rules are found in D.



Rules


<age:20..24>
<income:250..499>
<children:0..0>
<children:1..1>
<married:1>
<income:250..499>
<age:20..24>
<children:0..0>
<income:250..499>
<income:250..499>
<children:1..1>
<married:1>
<age:20..24>
<income:250..499>
<children:0..0>
<married:1>
<children:1..1>
<income:250..499>

Problems with the mapping approach

- *The partitioning problem*
  - The rules generated depend heavily on how the quantitative attributes are partitioned (or discretized).
- *The fragmented rules problem*
  - Some of the rules generated can be combined to form more concise rules.

The partitioning problem

- The partitioning of a quantitative attribute into intervals cannot be too fine or too coarse.
- Example: if "age" is partitioned into 20 intervals, each containing a single value (i.e., [20..20], [21..21], …, etc.) then no itemset (in our toy example *D*) concerning "age" is frequent. Hence, no rules concerning "age" can be derived.
- The items are too *specific*. They don't get enough supports to derive rules.

The partitioning problem

- Example: if "income" is partitioned into only 1 interval (i.e., <income:250..1000>), then many "obvious" rules will be generated.
- For example:

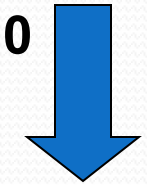
- The items are too *general* to be useful.

The partitioning problem

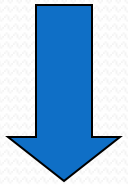
- To avoid partitioning the intervals too fine, one method is to specify a maximum support (maxsup) parameter. Intervals are allowed to merge with neighboring intervals as long as the resulting support does not exceed maxsup.
- Similar to equal-depth discretization with a controlled “depth” (maxsup)
- Example: suppose “age” is partitioned into intervals of size 2, the support counts of the intervals are:

item	{<age:20..21>}	{<age:22..23>}	{<age:24..25>}	{<age:26..27>}	{<age:28..29>}
Support count	2	2	2	2	2
item	{<age:30..31>}	{<age:32..33>}	{<age:34..35>}	{<age:36..37>}	{<age:38..39>}
Support count	2	2	4	4	3

suppose maxsup is set to 10



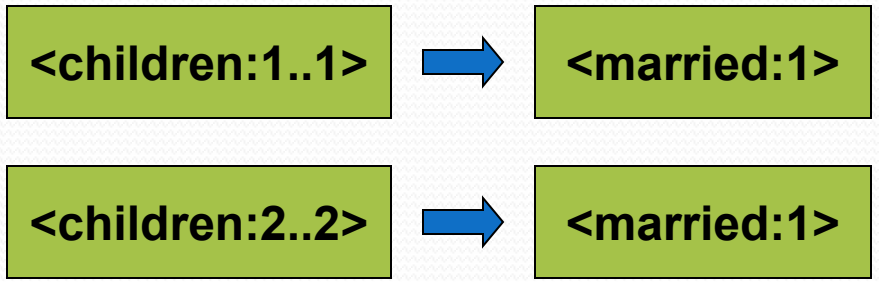
item	{<age:20..23>}	{<age:24..27>}	{<age:28..31>}	{<age:32..35>}	{<age:36..39>}
Support count	4	4	4	6	7



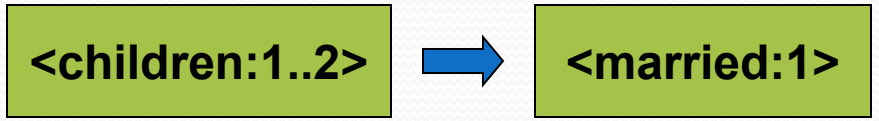
item	{<age:20..27>}	{<age:28..35>}	{<age:36..39>}
Support count	8	10	7

Fragmented rules

- Consider the following rules that were generated in our example:



- These rules could be combined into:



Note that the combined rule would never be generated in our example because the item <children:1..2> is not created during the partition process (not in I_R).

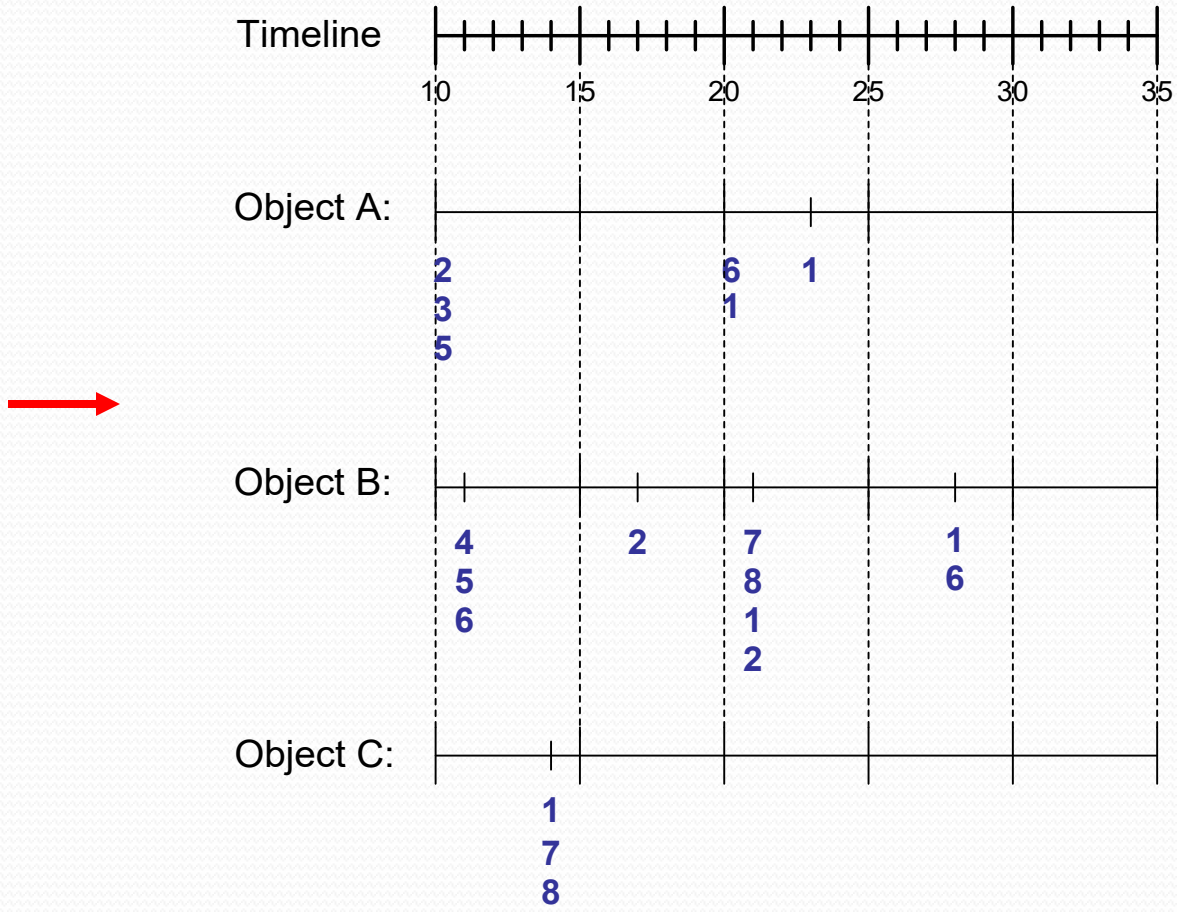
Fragmented rules

- Rule merging needs to be done to handle the fragmented rule problem.
- Rules that share the same right-hand side and have the same set of attributes on the left-hand side, should be considered for possible merging.

Sequence Data

Sequence Database:

Object	Timestamp	Items
A	10	2, 3, 5
A	20	6, 1
A	23	1
B	11	4, 5, 6
B	17	2
B	21	7, 8, 1, 2
B	28	1, 6
C	14	1, 8, 7



Example: Customers (objects) who buy sets of products (itemsets) at different timestamps

Formal Definition of a Sequence

- A sequence *s* is an ordered list of transactions

$$s = < t_1\ t_2\ t_3\ \ldots >$$

Each transaction t is a collection of items

$$t = \{i_1, i_2, \ldots, i_k\}$$

Each transaction is associated with a timestamp

Length of a sequence, |s|, is the number of transactions in the sequence

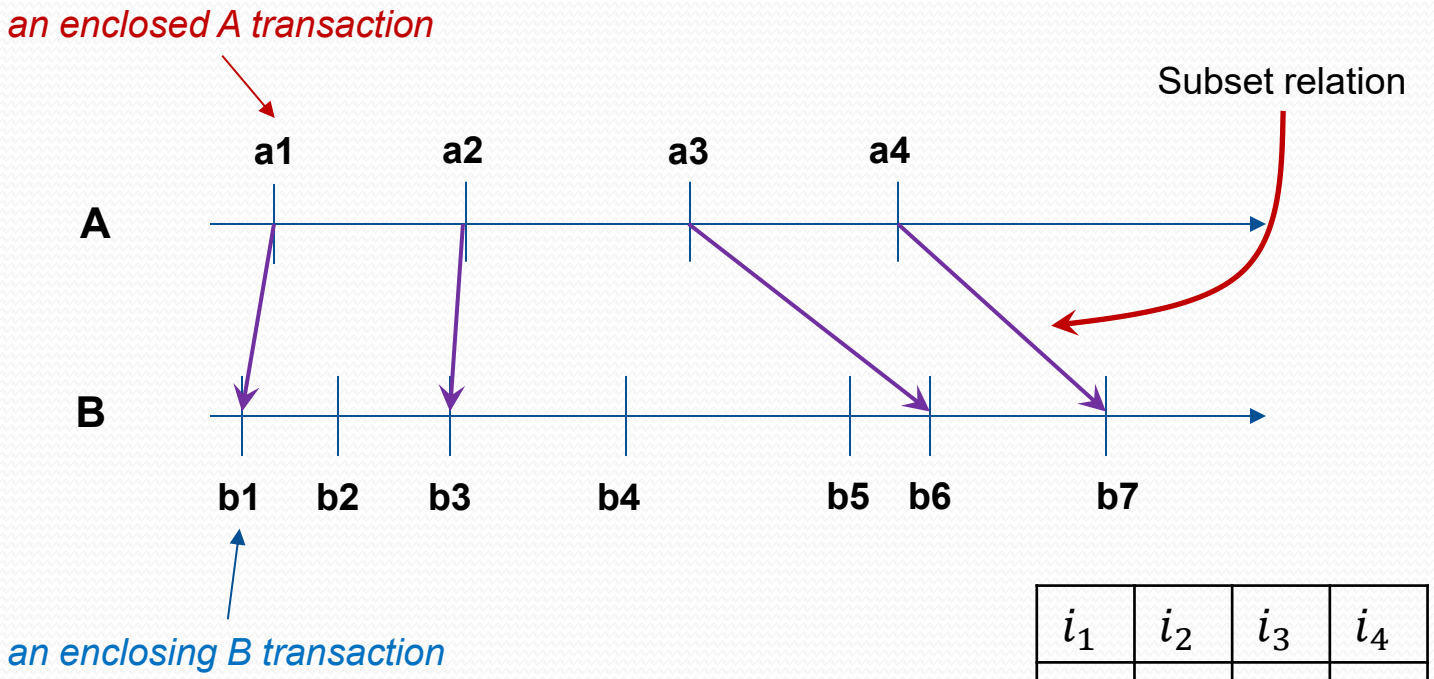
- A *k*-sequence is a sequence that contains *k* items
- Ex. of 3-sequences: <{a,b}, {a}>, <{a,b,c}>, <{a},{b},{b}>

length: 2 1 3

Definition

- A sequence $<a_1\ a_2\ \ldots\ a_n>$ is contained in another sequence $<b_1\ b_2\ \ldots\ b_m>$ $(m \geq n)$ if there exist integers (indices into the b's sequence) $i_1 < i_2 < \ldots < i_n$ such that $a_1 \subseteq b_{i1}$, $a_2 \subseteq b_{i2}$, …, $a_n \subseteq b_{in}$
- The support of a subsequence w is defined as the fraction of data sequences that contain w
- A sequential pattern is a frequent subsequence (i.e., a subsequence whose support is $\geq \rho_s$)

Illustration: Sequence A contained in Sequence B



i_1	i_2	i_3	i_4
1	3	6	7

Indices of the enclosing B transactions

Definition

In the following 3 cases, does Sequence B contain Sequence A?

Data sequence (B)	Subsequence (A)	Contain?
< {2,4} {3,5,6} {8} >	< {2} {3,5} >	Yes
< {1,2} {3,4} >	< {1} {2} >	No
< {2,4} {2,4} {2,5} >	< {2} {4} >	Yes

Definition

- Given:
  - a database of sequences
  - a user-specified minimum support threshold, ρ_s
- Task:
  - Find all subsequences with support ≥ ρ_s

Sequential Pattern Mining: Example

Sequence A is: <{1, 2, 4}, {2, 3}, {5}>

Object	Timestamp	Items
A	1	1,2,4
A	2	2,3
A	3	5
B	1	1,2
B	2	2,3,4
C	1	1, 2
C	2	2,3,4
C	3	2,4,5
D	1	2
D	2	3, 4
D	3	4, 5
E	1	1, 3
E	2	2, 4, 5

db records (5 data sequences)

ρ_s = 50%

Examples of Frequent Subsequences:

< {1,2} >	s=60%
< {2,3} >	s=60%
< {2,4}>	s=80%
< {3} {5}>	s=80%
< {1} {2} >	s=80%
< {2} {2} >	s=60%
< {1} {2,3} >	s=60%
< {2} {2,3} >	s=60%
< {1,2} {2,3} >	s=60%

supported by sequences A, C, D, E

Sequential Pattern Mining: Example

Sequence A is: <{1, 2, 4}, {2, 3}, {5}>

Object	Timestamp	Items
A	1	1,2,4
A	2	2,3
A	3	5
B	1	1,2
B	2	2,3,4
C	1	1, 2
C	2	2,3,4
C	3	2,4,5
D	1	2
D	2	3, 4
D	3	4, 5
E	1	1, 3
E	2	2, 4, 5

ρ_s = 50%

Examples of Frequent Subsequences:

< {1,2} >	s=60%
< {2,3} >	s=60%
< {2,4}>	s=80%
< {3} {5}>	s=80%
< {1} {2} >	s=80%
< {2} {2} >	s=60%
< {1} {2,3} >	s=60%
< {2} {2,3} >	s=60%
< {1,2} {2,3} >	s=60%

Supported by A, B, C, but not D, E

Generalized Sequential Pattern (GSP)

- Step 1:

  Make the first pass over the sequence database D to yield all 1-item frequent sequences

- Step 2:

Repeat until no new frequent sequences are found. During the k-th iteration:

Candidate Generation:

Merge pairs of frequent subsequences found in the (k-1)st pass to generate candidate sequences that contain k items

Candidate Pruning:

Prune candidate k-sequences that contain infrequent (k-1)-subsequences

Support Counting:

Make a new pass over the sequence database D to find the support for these candidate sequences

Candidate Generation

- Base case (k=2):
  - Merging two frequent 1-sequences
    $< \{i_1\} >$ and $< \{i_2\} >$ will produce 5 candidate 2-sequences:

$$< \{i_1\}\,\{i_2\} > , < \{i_2\}\,\{i_1\} > , < \{i_1\}\,\{i_1\} > , < \{i_2\}\,\{i_2\} >, < \{i_1\ i_2\} >$$

Candidate Generation

- General case (k>2):
  - Merge two frequent (k-1)-sequences w_1 and w_2 to produce a candidate k-sequence if the subsequence obtained by removing the first item in w_1 is the same as the subsequence obtained by removing the last item in w_2
  - The resulting candidate after merging is given by the sequence w_1 extended with the last item of w_2.
    - If the last two items in w_2 belong to the same transaction, then the last item in w_2 becomes part of the last transaction in w_1
    - Otherwise, the last item in w_2 becomes a separate transaction appended to the end of w_1

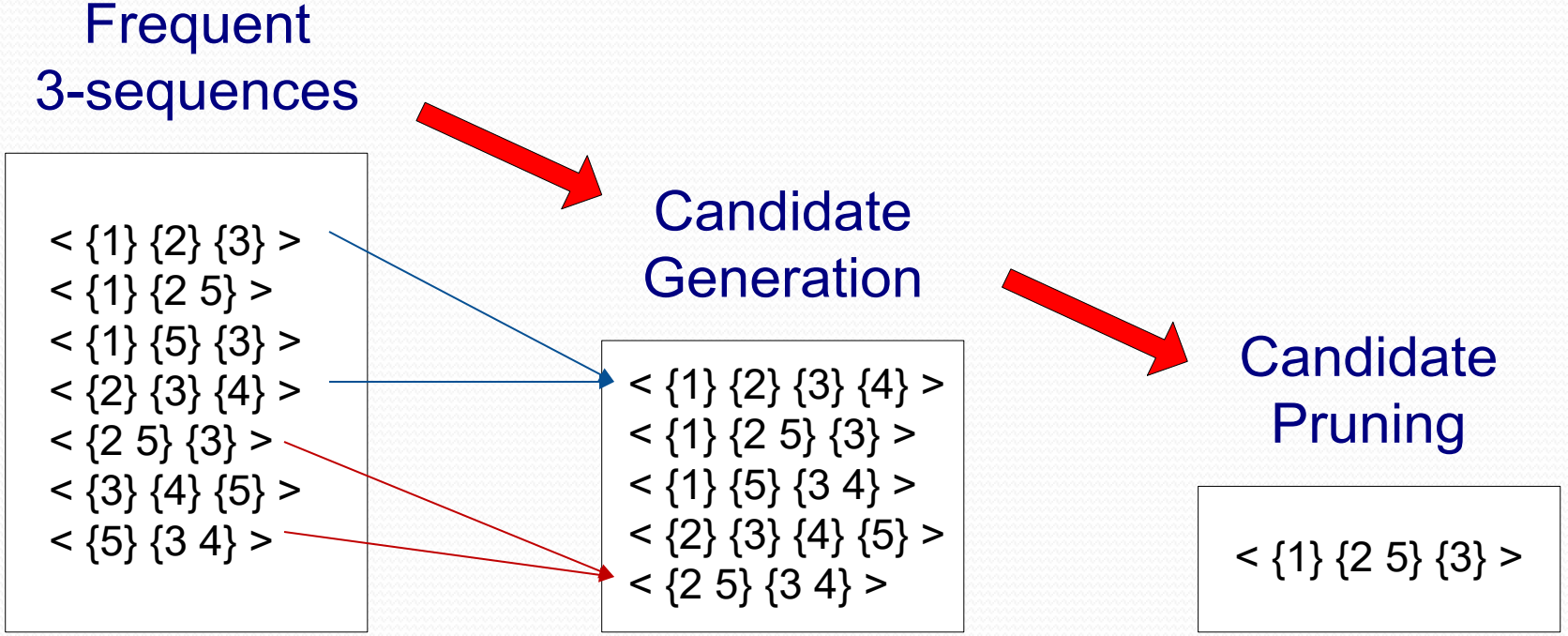
Candidate Generation Examples

- Merging the sequences
  w_1 =<{1} {2 3} {4}> and w_2 =<{2 3} {4 5}>
  will produce the candidate sequence < {1} {2 3} {4 5}> because the last two items in w_2 (4 and 5) belong to the same transaction

- Merging the sequences
  w_1 =<{1} {2 3} {4}> and w_2 =<{2 3} {4} {5}>
  will produce the candidate sequence < {1} {2 3} {4} {5}> because the last two items in w_2 (4 and 5) do not belong to the same transaction

- We do not have to merge the sequences
  w_1 =<{1} {2 6} {4}> and w_2 =<{1} {2} {4 5}>

GSP Example


Frequent
3-sequences
< {1} {2} {3} >
< {1} {2 5} >
< {1} {5} {3} >
< {2} {3} {4} >
< {2 5} {3} >
< {3} {4} {5} >
< {5} {3 4} >
Candidate
Generation
< {1} {2} {3} {4} >
< {1} {2 5} {3} >
< {1} {5} {3 4} >
< {2} {3} {4} {5} >
< {2 5} {3 4} >
Candidate
Pruning
< {1} {2 5} {3} >

Summary

- Association rule mining is an important data mining task
- Frequent itemset mining is the main component of association rules mining
  - association rules can be generated from frequent patterns
- The basic Apriori algorithm and some of its optimized versions
- FP-growth algorithm
- Lift as an interestingness measure of rules
- Quantitative association rules mining and frequent sequences mining can be done with the Apriori property.